

# 文件

## 第10章

# 主要内容

## 10.1 文件概述

## 10.2 Python的文件对象

## 10.3 文本文件的操作

## 10.4 文件和目录操作(可选内容)

# 文件：二进制和文本文件

按照数据的组织形式，文件可分为文本文件和二进制文件

- 二进制文件
  - 存储的是一个一个的**字节**，无法用常规的文本编辑器读写
  - 图像文件、可执行文件、音视频文件、Office文档等
- 文本文件
  - 存储文本**字符串**，由**若干行**组成，行之间以**行分隔符**(EOL, linebreak)隔开
  - 可通过文本编辑器等正常显示、编辑，人类能够直接阅读和理解
  - 文本文件里面的字符采用某种**编码方式**编码后保存到存储设备上
- 文本文件的行分隔符：可通过os.linesep查看
  - 不同操作系统使用的行分隔符可能各不一样(\r, \r\n或者\n)
  - python解释器在进行**文本文件的读写**时统一用**\n作为行分隔符**，会自动进行相应的转换



# 文件和目录

绝对路径名: C:\Users\tom\AppData\Roaming\Python\Python36\site-packages  
相对路径名: pj2.py          projects\pj1.py

- 文件以目录（文件夹）的方式组织成目录树
  - 一个目录可包含多个文件，也可以包含子目录，这些子目录再包含文件或子目录
- 路径名(pathname): 从目录树中**某个起点**开始直到结束位置，途中经过的节点名字组合在一起，之间通过**路径分隔符**隔开
  - Linux和MacOS系统的路径分隔符为'/'，而微软系统中为r'\'
  - **在实践中都可以使用/分割路径名，python会转换为操作系统所采用的路径分隔符**
- **绝对路径名**: 以**目录树中的根开始**的路径名，也就是以路径分隔符开始
- **相对路径名**: 不以目录树中的根开始的路径名
  - 相对路径名从**工作目录**开始: <cwd> + relative-pathname
  - 任何正在执行的程序都有一个工作目录
    - os模块的getcwd可查看当前工作目录。chdir可改变工作目录
  - 集成开发环境IDLE执行py程序时会把工作目录设置为py程序所在的目录:
  - VS Code: 设置中搜索executeinfiledir, 找到扩展python的配置(Terminal: Execute In File Dir) 后勾选, 在执行时切换到文件所在目录。 "python.terminal.executeInFileDir": true
- 每个目录中有2个特殊的目录名, 其中 **..**(两个点)表示当前目录的**父目录**

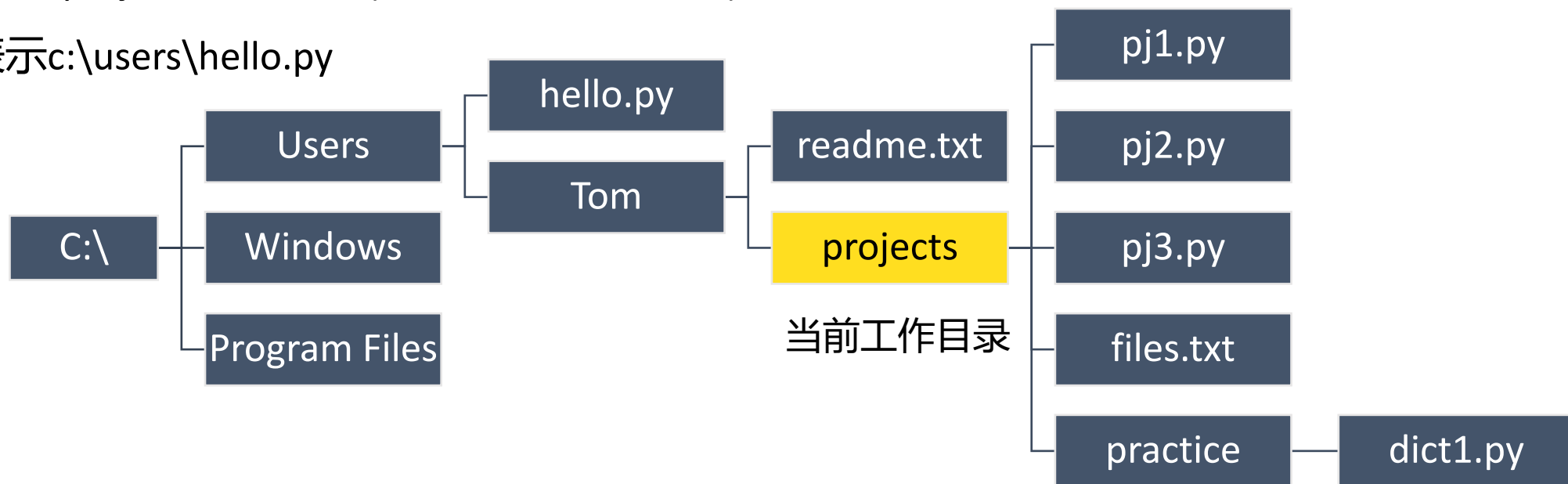
```
>>> import os
>>> os.getcwd()
'C:\\Users\\Tom\\projects'
>>> os.chdir('..')
```

# 文件和目录：工作目录和相对路径

假设当前工作目录为projects:

```
>>> import os
>>> os.getcwd()
'C:\\Users\\Tom\\projects'
```

- files.txt 表示projects下的files.txt
- practice\\dict1.py表示practice子目录下的dict1.py
- ../readme.txt 表示projects的父目录(这里为c:\\users\\Tom)中的readme.txt
- ../../hello.py 表示c:\\users\\hello.py



- 绝对路径：你的应用程序移动到任何位置都可以访问。如果复制到另一台机器，绝对路径可能有问题
- **相对路径**：你的应用程序和文件可以**整体移动**到另外一个位置、另一台机器，还能够访问

# 主要内容

## 10.1 文件概述

## 10.2 Python的文件对象

## 10.3 文本文件的操作

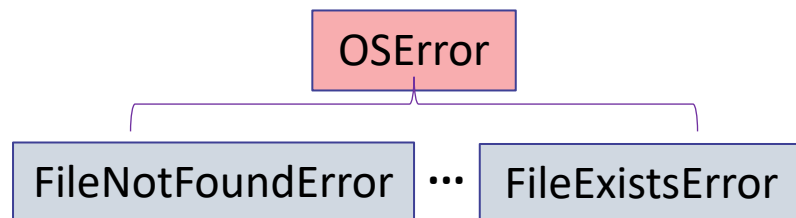
## 10.4 文件和目录操作(可选内容)

# 文件对象：打开文件

`open(file, mode='r', buffering=-1, encoding=None, errors=None, newline=None, ...)`

以指定模式打开相应的文件，返回某种类型的文件对象。出错时抛出异常**OSError或者OSError的某个子类型**

- **文件名file**: 可采用相对路径或绝对路径
- **访问模式(mode)**: 第2个参数，打开文件后的处理方式
- **buffering**: 指定缓存策略。交互式文本文件缺省采用行缓存，而普通文本模式和二进制模式缺省采用固定大小的块缓存
- **编码方式(encoding)**: 以文本方式打开时所采用的编码方式
  - 可取值: utf-8, ascii, gbk, gb2312, big5等
  - 缺省**locale.getencoding()**返回的编码方式: 如果为'cp936', 表示GBK
  - python源程序缺省采用utf-8编码方式
- **errors**: 文本文件读写时编码或解码出现错误时怎样处理。可取值'strict', 'ignore', 'replace'等
  - 缺省'strict', 抛出异常，编码时的异常为UnicodeEncodeError，解码时异常为UnicodeDecodeError
  - 'ignore', 忽略后继续
  - 'replace', 替代为特定字符。编码(写)时替代为?, 解码(读)时替代为\uFFFD, 即❖, '**\N{REPLACEMENT CHARACTER}**'
- **newline**: 决定如何处理行分隔符。缺省自动识别并将其转换为'\n'



# 文件对象：访问模式

```
>>> import os
>>> os.path.exists('tmp.txt')
True
```

open(file, mode='r', buffering=-1, encoding=None, errors=None, newline=None, ...)

访问模式(mode)：第2个参数，打开文件后的处理方式：可以执行的**操作**（读，写，还是读写）和**文件指针**

- 可选的文件类型：文本(text)或二进制(binary)模式，t或b。缺省为t
- rwxa必须指定一个，且只能指定一个
- 缺省'r'，相当于'rt'，即打开文本文件读
- 可选的+，表示读写模式，需要指定rwxa中的某一个
- 操作系统维护**文件指针**，给出下一次的读写操作所对应的位置
  - a模式打开时，指针为文件尾部。其他模式打开时指针为0

```
以读模式打开文本文件 f = open('a.txt')
以写模式打开文本文件 f = open('a.txt', 'w')
以附加模式打开文本文件写 f = open('a.txt', 'a')
以读模式打开文本文件，编码方式为'utf-8'
f = open('hello.py', encoding='utf-8')
```

```
以读模式打开二进制文件
f = open('a.txt', 'rb')
以写模式打开二进制文件
f = open('a.txt', 'wb')

以读写模式打开文本文件
f = open('a.txt', 'r+')
```

| 值 | 说明                                              | 文件指针   |
|---|-------------------------------------------------|--------|
| r | 读模式，不存在时异常FileNotFoundError                     | 0      |
| w | 写模式，文件存在时会先 <b>截取覆盖原有文件!!!</b> 不存在时创建           | 0      |
| x | 写模式，文件存在时抛异常FileExistsError，不存在时创建              | 0      |
| a | <b>附加模式</b> ，打开写，不存在时会创建，存在时会附加到文件尾部            | 文件尾部   |
| + | <b>读写模式，较少使用！</b> 可使用r+ w+ x+ a+，注意w+会清空原有文件的内容 | 参见rwxa |
| t | 文本模式，写入或读取字符串                                   |        |
| b | 二进制模式，写入或读取字节串(bytes类型)                         |        |



## 文件对象：打开文件

- 采用不同模式打开的文件对象的类型在io模块中定义
- 这些不同类型的文件对象**提供相同的接口**，但具体实现有所区别
  - 文件对象有多个属性，记录了文件打开时的一些参数
  - 文件对象有读写等方法，如果打开模式不允许这些方法，调用时抛出异常io.UnsupportedOperation

| 主要属性     | 描述                   |
|----------|----------------------|
| name     | 文件名                  |
| mode     | 打开模式                 |
| encoding | 编码方式，仅对文本模式有效        |
| errors   | 编解码出错时的处理方式，仅对文本模式有效 |
| closed   | 是否已经关闭               |

```
>>> f = open('sample.txt')
>>> f
<_io.TextIOWrapper name='sample.txt' mode='r'
encoding='cp936'>
>>> fw = open('sample.txt', 'wb')
>>> fw
<_io.BufferedWriter name='sample.txt'>
>>> fr = open('sample.txt', 'rb')
>>> fr
<_io.BufferedReader name='sample.txt'>
```

## 文件对象: 关闭文件

文件对象使用完毕后, 要及时关闭

- 释放文件资源, 将文件缓冲区中的数据写入文件, 确保数据完整性并避免可能的内存泄漏
- 第1种方式: 如果出现异常时, `f.close()`不会被调用, 导致文件资源不能释放
- 第2种方式(建议): 采用`with`语句, 任何情况 (没有异常, 出现异常已处理, 出现异常没有处理) 出现时都会确保文件对象被关闭

```
f = open(file)
process(f)
f.close()
```



```
with open(file) as f:
    process(f)
```

# with语句

- read\_file的实现有问题：文件不存在时或者存在但无权限时，无法打开文件，抛出异常FileNotFoundError或PermissionError，调用f.close()出错(NameError)

| 语句 | 文件存在且有权限 | 文件存在但编码方式错              | 文件存在无权限              | 文件不存在                  |
|----|----------|-------------------------|----------------------|------------------------|
| 1  | ✓        | ✓                       | ✗<br>PermissionError | ✗<br>FileNotFoundError |
| 2  | ✓        | ✗<br>UnicodeDecodeError |                      |                        |
| 3  |          |                         |                      | ✓                      |
| 4  |          |                         | ✓                    |                        |
| 5  | ✓        | ✓                       | ✗                    | ✗                      |
| 6  | ✓        |                         |                      |                        |
| 抛出 |          | UnicodeDecodeError      | NameError            | NameError              |

read\_file2(): 检查open是否成功

- locals(): 访问当前(local) 名字空间的名字

```
def read_file(pathname):  
    try:  
        f = open(pathname) #1  
        return f.readlines() #2  
    except FileNotFoundError as e: #3  
        print(type(e), e)  
    except OSError as e: #4  
        print(type(e), e)  
    finally: #5  
        f.close()  
  
    print('end') #6
```

```
def read_file2(pathname):  
    try:  
        f = open(pathname) #1  
        return f.readlines() #2  
    except OSError as e: #3  
        print(type(e), e)  
    finally:  
        if 'f' in locals():  
            f.close()  
  
    print('end') #4
```

## with语句 (10.2.3 三种文件操作方式)

with语句用于上下文管理:

- 如果对象支持上下文管理协议, 则可用于with语句
  - 两个魔术方法: `__enter__()`, `__exit__()`
- 执行with block的代码
  - 执行完成后, 不管异常是否出现都会调用对象的`__exit__()`结束上下文, 释放资源
  - 对于文件对象而言, with fileobj: body, 最后会关闭该文件

```
def read_file3(pathname):
    try:
        with open(pathname) as f:
            return f.readlines()
    except FileNotFoundError as e:
        print(type(e), e)
    except OSError as e:
        print(type(e), e)
    except UnicodeDecodeError:
        with open(pathname, encoding="utf-8") as f:
            return f.readlines()
```

read\_file.py

```
with context_expr [as obj]:
    with_block
```

```
context_obj = context_expr
obj = context_obj.__enter__()
try:
    with_block
finally:
    context_obj.__exit__()
```

```
class LowerCaseFilter:
    def __init__(self, file):
        self.file = file

    def __enter__(self):
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        self.file.close()

    def __iter__(self):
        for line in self.file:
            yield line.lower()
```

lower\_case\_filter.py

```
with LowerCaseFilter(file) as f:
    for line in f:
        print(line, end='')
```

# with语句

- Python3.10, with语句支持多个上下文管理器, 即:

```
with context_expr [as obj]:  
    with_block
```

```
with context_expr1 [as obj1]:  
    with context_expr2 [as obj2]:  
        with_block
```



```
with (context_expr1 [as obj1],  
      context_expr2 [as obj2]) :  
    with_block
```

```
def tee(in_file, out_file1, out_file2):  
    with (open(in_file, 'rb') as file,  
          open(out_file1, 'wb') as file1,  
          open(out_file2, 'wb') as file2):  
        text = file.read()  
        file1.write(text)  
        file2.write(text)
```

tee\_file.py

# 主要内容

10.1 文件概述

10.2 Python的文件对象

**10.3 文本文件的操作**

10.4 文件和目录操作(可选内容)

# 文本文件对象的常用操作

|    | 方法                       | 说明                                                                         |
|----|--------------------------|----------------------------------------------------------------------------|
| 读  | f.read([size=-1])        | 读取最大size个字符或EOF，size为-1时表示读取直到EOF                                          |
|    | f.readline()             | 读取字符直到遇到换行符或者EOF，如果读时马上遇到EOF，则返回空字符串。<br>注意在 <b>读时遇到行分隔符时，\n也包括在内</b>      |
|    | f.readlines()            | 读取直到EOF，返回一个列表，每个元素为读到的每行内容 <b>(包括行尾换行符\n)</b>                             |
|    | next(f)                  | 以读方式打开的文件是一个 <b>迭代器</b> ，下一个元素为文件中下一行的内容 <b>(包括\n)</b> ，直到EOF为止            |
| 写  | f.write(text)            | 将字符串text写入文件，返回写入的字符个数                                                     |
|    | f.writelines(lines)      | 把lines中的每个元素逐个写入文件，要求list的元素必须为字符串，返回None                                  |
| 其他 | f.flush()                | 缓冲区的内容同步到存储设备                                                              |
|    | f.close()                | 关闭文件，释放资源，写模式下还同步缓冲区内容到存储设备                                                |
|    | f.truncate([size])       | 文件截取后长度变为size，缺省为当前文件指针位置(文件打开时初始为0)，返回文件的当前长度                             |
|    | f.tell()                 | 访问打开文件的当前指针位置（字节为单位）                                                       |
|    | f.seek(offset, whence=0) | 移动文件指针位置到基准位置的offset，缺省whence=0，表示从头部开始，whence=1，表示当前指针位置，whence=2表示文件结尾开始 |

# 文本文件的写

- open时不指定编码方式时, 表示encoding缺省为[locale.getencoding\(\)](#)  
file = open('sample.txt')  
file = open('sample.txt', 'w')  
file = open('sample.py', 'w', encoding='utf-8')
- print(..., file=file): 不是写到标准输出, 而是指定的文件
- file.write(str\_obj): 将字符串的内容str\_obj按照打开文件时指定的编码方式转换为字节串后写入文件,  
**返回写入的字符个数**
- file.writelines(lines):
  - 传递的参数为一个**可迭代对象, 其中的元素为字符串**, 按照顺序将所有的字符串写入文件
  - 虽然名字叫writelines, 但可迭代对象的元素写入时**并不会自动添加换行符**
- file.flush(): 缓冲区的内容同步到存储设备
  - 写文件时, 内容暂存在缓冲区, 等待合适的时候会自动同步到存储设备

```
def write_lines(f, lines):  
    for line in lines:  
        f.write(line)
```

## 文本文件的写

- 建议采用with语句
- file.write()和file.writelines()都不会额外添加\n
- print()
  - 缺省会添加\n
  - 可通过关键字参数file传递文件对象，写入到对应的文件，而不是缺省的标准输出

```
def write_file0(file="sample.txt"):
    f = open(file, "w")
    print("encoding:%s" % f.encoding)
    f.write("line 1\n")
    f.write("line 2\n")
    f.close()
```

```
def write_file(file="sample.txt"):
    with open(file, "w") as f:
        f.write("[with] line 1\n")
        f.write("[with] line 2\n")
```

```
def write_file_writelines(file="sample.txt"):
    lines = "[writelines] line 1\n", "[writelines] line 2\n"
    with open(file, "a") as f:
        f.writelines(lines)
```

```
def write_file_print(file="sample.txt"):
    with open(file, "a") as f:
        print("[print] line 1", file=f)
        print("[print] line 2\n", end='', file=f)
```

# 文本文件的读

- read(size=-1) **返回读取的数据(字符串)**

- 不传递参数或者size<0时表示从当前位置读直到文件结束EOF (end of file)
- size>0: 从当前位置读指定数量的字符或文件结束EOF为止
- 如果读时已经为EOF, 返回**空字符串**

- readline(size=-1): 从当前位置读直到换行或文件结束, 返回读到的字符串, 如果遇到换行符, 则**换行也包括在内**

- 刚开始读遇到EOF时返回空字符串 ==> 可检查**返回值为空字符串**判断EOF
- 如果size >0, 表示最多读size个字符或者读到换行符或者读到EOF为止

- readlines(): 从文件读数据直到文件结束, 返回一个**字符串列表**, 其元素为读到的每行字符串 (包括换行符)

```
def read_file_read(file='sample.txt'):
    with open(file) as f:
        text = f.read()
        print(text, end='')
```

file\_read.py

```
def read_file_readline(file='sample.txt'):
    with open(file, 'r') as f:
        while True:
            line = f.readline()
            if not line:
                break
            print(line, end='')
```

```
def read_file_readlines(file='sample.txt'):
    with open(file, 'r') as f:
        line = f.readlines()
        print(line)
        print('-' * 20)
        for line in line:
            print(line, end='')
```

## 文本文件的读：以行为单位读

- 文件对象支持iterator协议，意味着调用`next(file)`获得下一行，即相当于`file.readline()`

```
def read_file_iterator(file='sample.txt'):
    with open(file, 'r') as f:
        for line in f:
            print(line, end='')
```

file\_read.py

```
with open('somefile.txt') as file:
    lines = list(file)
```

# 相当于调用

```
with open('somefile.txt') as file:
    lines = file.readlines()
```

```
with open(filename) as f:
    while True:
        line = f.readline()
        if not line:
            break
        process(line)
```

```
with open(filename) as f:
    lines = f.readlines()

for line in lines:
    process(line)
```

```
with open(filename) as f:
    text = f.read()

lines = text.splitlines()
for line in lines:
    process(line)
```

```
with open(filename) as f:
    for line in f:
        process(line)
```

- `readline()`: 以行为单位读时，遇到换行符时保留换行符
- `splitlines(keepends=False)`: 缺省不保留换行符

# 文本文件的读写:编码方式

- open不传递编码方式时缺省为系统采用的编码方式 (比如GBK)
- 解释器读取python源程序采用的编码方式缺省为UTF-8编码, 除非
  - 前面两行中通过coding指出了相应的编码方式

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
```

```
def read_file3(pathname):
    try:
        with open(pathname) as f:
            return f.readlines()
    except FileNotFoundError as e:
        print(type(e), e)
    except OSError as e:
        print(type(e), e)
    except UnicodeDecodeError:
        with open(pathname, encoding="utf-8") as f:
            return f.readlines()
```

read\_file.py

UTF-8编码的py文件用GBK方式打开后, 如果其中包含中文, 则读到中文时会报错

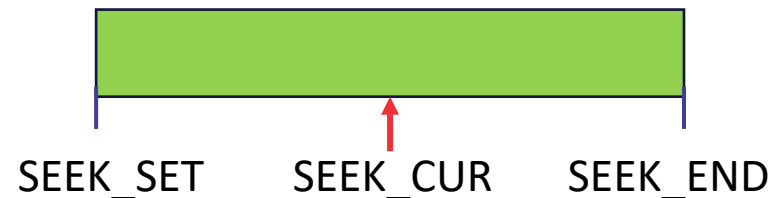
Exception <class '**UnicodeDecodeError**'> raised... 'gbk' codec can't decode byte

## 文本文件：移动文件指针

- `tell()`: 返回当前文件的指针所在位置(距离文件开始多少个字节)
- `seek(offset, whence=0)`
  - 将文件指针移动到文件的某个位置 (相对某个参考位置的偏移量, 以字节为单位)
  - `whence`给出了参考位置: 缺省为从文件开始处算起的第几个字节
    - `whence=0(os.SEEK_SET)`表示从文件开始
    - `whence=1(os.SEEK_CUR)`表示当前位置
    - `whence=2(os.SEEK_END)`表示文件结尾
- 对于文本方式的文件对象:
  - `whence=1`或`2`时, `offset`必须为`0`
  - `f.seek(0, 2)`: 移动到文件结尾

程序功能:

- 文件编码方式和大小
- 读取前面10个字符
- 读取文件中所有字符



```
def seek_file(file="sample.txt"):
    with open(file, "r") as f:
        print("encoding:", f.encoding)
        f.seek(0, os.SEEK_END)
        print(f'size:{f.tell()}')
        f.seek(0, os.SEEK_SET)
        firstN = f.read(10)
        print("current offset:", f.tell())
        f.seek(0)
        text = f.read()
        print(f"first 10 chars:\n{firstN}")
        print("-" * 40)
        print(text, end='')
```

file\_read.py

## 文本文件：文件读写示例

示例1：文件hw\_grades.csv中，保存了若干位同学某门课程的平时作业成绩(一行对应一位同学的作业成绩，均为整数)。对每位同学的作业成绩：按升序排列。处理后的结果保存到文件hw\_grades\_sorted.csv。

- csv (Comma-Seperated Value) 文件是一种文本文件类型，可以用常见的文本编辑器打开编辑，如：Windows系统自带的记事本软件
- 同时它也是一种简单的表格文件，可以用大多数表格软件打开编辑，如：Excel
- csv文件中每行文本的各个数据之间均以逗号分隔，即，逗号为各列数据的分隔符
- csv模块提供了对于csv文件的读写操作，不展开

hw\_grades.csv

```
99,5,86,84,60,65,65,37
7,32,37,85,93
47,43,29,84,63,0,16,23,60
6,66,42,88,14,50,31,86,31,85
57,95,4,40,41,87
91,48,46,28,58
13,44
55,51,96,88,76,30,25
91,31,90,72,23
58,70,34,74,87,6
```

# 文本文件：文件读写示例1

hw\_grades.csv - Excel

文件 开始 插入 绘图 页面布局 公式 数据 审阅 视图 帮助 Acrobat

A1 99

|    | A  | B  | C  | D  | E  | F  | G  | H  | I  | J  | K | L | M |
|----|----|----|----|----|----|----|----|----|----|----|---|---|---|
| 1  | 99 | 5  | 86 | 84 | 60 | 65 | 65 | 37 |    |    |   |   |   |
| 2  | 7  | 32 | 37 | 85 | 93 |    |    |    |    |    |   |   |   |
| 3  | 47 | 43 | 29 | 84 | 63 | 0  | 16 | 23 | 60 |    |   |   |   |
| 4  | 6  | 66 | 42 | 88 | 14 | 50 | 31 | 86 | 31 | 85 |   |   |   |
| 5  | 57 | 95 | 4  | 40 | 41 | 87 |    |    |    |    |   |   |   |
| 6  | 91 | 48 | 46 | 28 | 58 |    |    |    |    |    |   |   |   |
| 7  | 13 | 44 |    |    |    |    |    |    |    |    |   |   |   |
| 8  | 55 | 51 | 96 | 88 | 76 | 30 | 25 |    |    |    |   |   |   |
| 9  | 91 | 31 | 90 | 72 | 23 |    |    |    |    |    |   |   |   |
| 10 | 58 | 70 | 34 | 74 | 87 | 6  |    |    |    |    |   |   |   |
| 11 |    |    |    |    |    |    |    |    |    |    |   |   |   |
| 12 |    |    |    |    |    |    |    |    |    |    |   |   |   |
| 13 |    |    |    |    |    |    |    |    |    |    |   |   |   |
| 14 |    |    |    |    |    |    |    |    |    |    |   |   |   |
| 15 |    |    |    |    |    |    |    |    |    |    |   |   |   |
| 16 |    |    |    |    |    |    |    |    |    |    |   |   |   |

hw\_grades

就绪 辅助功能: 不可用

hw\_grades.csv

99,5,86,84,60,65,65,37  
7,32,37,85,93  
47,43,29,84,63,0,16,23,60  
6,66,42,88,14,50,31,86,31,85  
57,95,4,40,41,87  
91,48,46,28,58  
13,44  
55,51,96,88,76,30,25  
91,31,90,72,23  
58,70,34,74,87,6

# 文本文件：文件读写示例1

读.csv文件，处理后写入另一个文件

- 对每行进行操作：得到各个字段（整数），排序后输出

hw\_grade.py

```
def main():  
    with open("hw_grades.csv") as f:  
        data = f.readlines() # or list(f)  
  
    result = []  
    for line in data:  
        nums = line.strip().split(",") # 分割出各个整数字符串  
        nums = sorted(int(i) for i in nums) # 将每个整数字符串转换为整数后排序  
        # nums = sorted(map(int, nums))  
        nums = ",".join(str(i) for i in nums) + "\n" # 重新构成以逗号分隔的字符串  
        result.append(nums)  
  
    with open("hw_grades_n.txt", "w") as f:  
        f.writelines(result)  
  
if __name__ == "__main__":  
    main()
```

## 文本文件：文件读写示例2

示例2：为python源程序(假设为file.py)加上行号（出现在行首），保存在file\_n.py.txt中

```
import os

def number_line(pathname):
    with open(pathname, encoding="UTF-8") as file:
        lines = [f'{i:<3} {line}' for i, line in enumerate(file, 1)]

    root, ext = os.path.splitext(pathname)
    outfile = root + "_n" + ext + ".txt"

    with open(outfile, "w", encoding="UTF-8") as file:
        file.writelines(lines)

if __name__ == "__main__":
    number_line(__file__)
```

number\_line.py

## 拓展：二进制文件的读写

- open时mode包含字符b

```
file = open('sample.dat', 'rb')
file = open('sample.dat', 'wb')
```
- read(size=-1) **返回读取的数据(bytes对象)**
  - 不传递参数或者size<0时表示从当前位置读直到文件结束EOF
  - 从当前位置读指定数量的字节或文件结束EOF为止
  - 如果读时已经为EOF，返回**空bytes对象**
- write(bytes\_obj) **传递的是bytes对象**，将该对象中的数据写入文件，返回写入的字节数
- 二进制文件也支持writelines方法
- 也支持以行为单位读：readline(), readlines(), next(f)，即文件对象作为一个迭代器（元素为行）
  - 行：b'\n'表示一行的结束

```
def write_binary_file(file="sample.dat"):
    with open(file, "wb") as f:
        f.write(b"line 1\n")
        f.write(b"line 2: ")
        f.write(bytes(range(32)))# ASCII控制字符
        f.write(b"\n")
        f.write(b"line 3\n")
```

```
def read_binary_file(file="sample.dat"):
    with open(file, "rb") as f:
        text = f.read()
        print(text)
        f.seek(0)
        while True:
            line = f.readline()
            if not line:
                break
            print(line)
```

file\_binary.py

## 【拓展】 标准输入输出

`print(value1, value2, ..., sep=' ', end='\n', file=sys.stdout)`

- 一个程序运行时缺省会打开3个文件对象：
  - 设备也看成特殊的文件
  - `sys.stdin`, 标准输入, 缺省对应着键盘设备
    - `input()`: 从标准输入读取一行
  - `sys.stdout`, 标准输出, 缺省对应着显示器
    - `print()`: 向标准输出写
  - `sys.stderr`, 标准错误输出, 缺省对应着显示器
    - 用于输出错误信息
  - 这3个文件对象会自动打开, 也无需用户执行关闭操作
- 操作系统也提供相应的重定向机制, 允许运行程序时将这3个文件对象重定向其他设备或者文件

```
>>> import sys
>>> sys.stdout.write('hello world!\n')
hello world!
13
```

```
>>> file = open('tmp.txt', 'a')
>>> print('hello world', file=file)
>>> file.close()
```

`cmd <infile >outfile 2>errfile`

```
def my_input(prompt=' '):
    sys.stdout.write(prompt)
    sys.stdout.flush()
    return sys.stdin.readline().strip()
```

## 拓展: StringIO和BytesIO

- io模块的StringIO和BytesIO允许通过文件的操作方式来实现对于内存缓冲区的读写
- StringIO相当于以文本方式读写: `io.StringIO(initial_value='')`, 参数为初始内容
- BytesIO相当于以二进制方式读写: `io.BytesIO(initial_bytes=b'')`
- StringIO和ByteIO对象的`getvalue()`: 获得目前内存缓冲区的值

```
import io
def stringio_demo():
    file = io.StringIO("line 1\nline 2\n")
    while True:
        line = file.readline()
        if not line:
            break
        print(line, end='')
    file.write("line 3\nline 4\n")
    print(file.getvalue())
    file.seek(0)
    for line in file:
        print(line, end='')
```

stringio.py

```
def bytesio_demo():
    file = io.BytesIO()
    file.write(b"line 1\nline 2\n")
    print(file.getvalue())
```

# 主要内容

## 10.1 文件概述

## 10.2 Python的文件对象

## 10.3 文本文件的操作

## 10.4 **文件和目录操作(可选内容)**

# 操作文件的不同层次

- 读写文件内容: 文件对象
- 文件和目录操作: `os`, `os.path`
- 文件的高级操作: `shutil`

关于文件和目录的操作，分布在多个模块中，包括`os`、`os.path`、`glob`和`shutil`。python3.4引入`pathlib`模块，引入`Path`对象，以面向对象的方式处理文件系统路径，将这些文件、目录相关的操作有机地整合在一起。

# 13.4.1 OS(Operating System)模块

| 方法                     | 功能说明                                                             |
|------------------------|------------------------------------------------------------------|
| os.remove(path)        | 删除文件                                                             |
| os.rename(src, dst)    | 重命名文件或目录                                                         |
| os.mkdir(path)         | 创建目录                                                             |
| os.rmdir(path)         | 删除空目录                                                            |
| os.getcwd()            | 获取当前工作目录                                                         |
| os.chdir(path)         | 改变当前工作目录                                                         |
| os.listdir(path)       | 返回指定目录下的所有文件和目录名                                                 |
| os.scandir(path)       | 迭代器，os.DirEntry，支持的属性和方法包括name, path, isdir(), isfile()等         |
| os.walk(top)           | 遍历目录，迭代器的元素为(dirpath, dirnames, filenames)，给出目录路径名，该目录中的子目录名和文件名 |
| os.path.exists(path)   | 检查文件或目录是否存在                                                      |
| os.path.isfile(path)   | 检查给定路径是否为文件                                                      |
| os.path.isdir(path)    | 检查给定路径是否为目录                                                      |
| os.path.split(path)    | 将path分割为dirname和basename                                         |
| os.path.splitext(path) | 将path分割为root和exit                                                |

```
import os
file_path = 'example.txt'
if os.path.exists(file_path):
    print(f"文件{file_path}存在")
else:
    print(f"文件{file_path}不存在")
```

# 13.4.2 shutil(Shell Utility)模块

| 方法                                                                                  | 功能说明                          |
|-------------------------------------------------------------------------------------|-------------------------------|
| shutil.copy(src, dst)                                                               | 复制文件src内容+权限， dst可以是目录        |
| shutil.copy2(src, dst)                                                              | 复制文件src内容+状态， dst可以是目录        |
| shutil.copytree(src, dst, ignore=None, copy_function=copy2, dirs_exist_ok=False...) | 递归复制src开始的整个目录树到dst（不是复制入dst） |
| shutil.move(src, dst)                                                               | 移动文件或目录src到dst                |
| shutil.rmtree(path)                                                                 | 递归删除目录及其内容                    |
| shutil.make_archive(base_name, format, root_dir=None, base_dir=None)                | 创建压缩归档文件，返回归档文件路径名            |
| shutil.unpack_archive(filename, extract_dir=None)                                   | 解压缩归档文件                       |

```
>>> import os, shutil
>>> os.mkdir('temp')
>>> os.chdir('temp') # 切换目录到temp
>>> shutil.make_archive('windows.etc', 'gztar', root_dir=r'C:\Windows\System32\drivers', base_dir='etc')
'...\temp\windows.etc.tar.gz'
>>> shutil.make_archive('windows.etc2', 'gztar', base_dir=r'C:\Windows\System32\drivers\etc')
'windows.etc2.tar.gz'
>>> shutil.unpack_archive('windows.etc.tar.gz', 'temp2') # 展开在temp2目录
>>> shutil.rmtree('temp2') # 将temp2下所有目录和文件全部删除
```

- format的主要取值：zip, tar, gztar等
- root\_dir：切换到该目录后归档
- base\_dir：要归档内容的目录
- root\_dir和base\_dir缺省为当前目录
- base\_dir为绝对路径时，root\_dir无意义
- base\_dir为相对路径时，相对于root\_dir
- 指定了root\_dir时返回绝对路径名

- base\_dir = /absolute/path/name, archive里面出现的目录结构: absolute/path/name
- base\_dir = relative/path/name, archive里面出现的目录结构: relative/path/name

### 13.4.3 文件和目录操作实例

[例] 列出当前目录下，所有扩展名为.py的文件.

os\_demo.py

```
def list_file_ext(ext):  
    file_list = [fn for fn in os.listdir('.')  
                  if os.path.isfile(fn) and fn.endswith(ext)]  
    print(file_list)
```

glob模块：文件通配符匹配

- glob(pathname): 返回与pathname匹配的所有文件的路径名组成的列表
- iglob(pathname): 与glob类似，只是返回的是一个迭代器

```
file_list = glob.glob('*.' + ext)
```

### 13.4.3 文件和目录操作实例

[例] 将当前目录下所有扩展名为.txt的文件更改为.tx

os\_demo.py

```
def rename_extention(old, new):  
    file_list = [fn for fn in os.listdir() if fn.endswith(old)]  
    for fn in file_list:  
        newname = fn[: -len(old)] + new # fn.removesuffix(old) + new  
        os.rename(fn, newname)  
        print(fn + "更改为: " + newname)
```

## 13.4.3 文件和目录操作实例

[例] 归档当前目录

os\_demo.py

```
def archive_current_dir(override=True):
    """归档到当前目录的父目录的<dir>.zip """
    basename = os.path.basename(os.getcwd())
    parent = os.path.dirname(os.getcwd())
    archive_name = basename + ".zip"
    archive_pathname = os.path.join(parent, archive_name)
    if os.path.exists(archive_pathname):
        if not override:
            raise FileExistsError(f"{archive_pathname} already exists!")
        os.remove(archive_pathname)

    pathname = os.path.join(parent, basename)
    shutil.make_archive(pathname, 'zip', root_dir='..', base_dir=basename)
    print(f'archive_name = {archive_name}, {archive_pathname = }')
```

## 13.4.3 文件和目录操作实例

[例] 列出指定目录下（包括其各级子目录）所有扩展名为.py或.txt的文件

```
def visit_tree(path, include=None):
    """递归遍历目录树"""
    if not os.path.isdir(path):
        raise ValueError(f"{path}...")
    names = []
    with os.scandir(path) as entries:
        for entry in entries:
            if entry.is_dir():
                names.extend(visit_tree(entry.path))
            elif entry.is_file():
                if not include or include(entry.name):
                    names.append(entry.path)
    return names
```

```
def visit_tree2(path, include=None):
    """递归遍历目录树"""
    if not os.path.isdir(path):
        raise ValueError(f"{path}...")
    names = []
    for name in os.listdir(path):
        filename = os.path.join(path, name)
        if os.path.isdir(filename):
            names.extend(visit_tree2(filename, include))
        elif os.path.isfile(filename):
            if not include or include(name):
                names.append(filename)
    return names
```

walk\_demo.py

```
def include_func(name):
    """返回True表示包含该文件"""
    return name.endswith(".py") or name.endswith(".txt")
```

### 13.4.3 文件和目录操作实例

[例] 列出指定目录下（包括其各级子目录）所有扩展名为.py或.txt的文件

```
def visit_tree3(path, include=None):
    if not os.path.isdir(path):
        raise ValueError(f"{path} is not a directory or does not exist.")

    names = []
    # os.walk返回的对象的每个元素为3个元素的元组：目录、该目录下的子目录列表和文件列表
    for root, dirs, files in os.walk(path):
        #         for dir in dirs:
        #             print(os.path.join(root, dir))
        for file in files:
            filename = os.path.join(root, file)
            if not include or include(file):
                names.append(filename)
    return names
```

walk\_demo.py